

Ejercicio Demostrativo

Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 25 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Estructura de Datos - C] hola

1. Marco Formativo

1.1 Valores

Responsabilidad	El estudiante es responsable de comprender el procedimiento de cada algoritmo demostrado, trazar manualmente los pasos antes de codificarlos y verificar que su implementación produce el arreglo ordenado correcto.
------------------------	--

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Implementa estructuras de datos lineales y no lineales usando programación orientada a objetos para desarrollar soluciones eficientes.
Competencia Específica	Implementa y traza manualmente los algoritmos de ordenamiento Inserción, Selección y Burbuja sobre arreglos; aplica búsqueda lineal y binaria; e implementa una Matriz Dispersa mediante lista enlazada con cabeceras, comprendiendo las ventajas de memoria frente a una matriz densa.

2. Palabras Clave

Algoritmo de ordenamiento: Procedimiento que reorganiza los elementos de un arreglo o lista en un orden específico (ascendente, descendente, alfabético, numérico).

Insertion Sort (Inserción): Divide el arreglo en una parte ordenada y otra desordenada; toma elementos de la parte desordenada y los ubica en su posición correcta dentro de la parte ordenada.

Selection Sort (Selección): Busca el elemento mínimo del subarray desordenado y lo intercambia con la primera posición de ese subarray, repitiendo hasta ordenar todo.

Bubble Sort (Burbuja): Compara pares de elementos adyacentes e intercambia aquellos que estén en orden incorrecto, repitiendo pasadas hasta que no haya más intercambios.

Complejidad temporal: Medida de cuánto tiempo tarda un algoritmo en función del tamaño de entrada n . Se expresa con notación Big-O ($O(n^2)$, $O(n \log n)$, $O(n)$).

Búsqueda lineal: Recorre el arreglo elemento por elemento desde el inicio hasta encontrar el objetivo. Complejidad $O(n)$.

Búsqueda binaria: Solo funciona sobre arreglos ordenados; divide el espacio de búsqueda a la mitad en cada paso. Complejidad $O(\log n)$.

Matriz dispersa: Matriz en la que la mayoría de sus elementos son cero. Almacenar todos los ceros desperdicia memoria; se representa eficientemente guardando solo los valores no nulos.

Implementación con lista enlazada: Representa la matriz dispersa como una cadena de nodos, cada uno con (valor, fila, columna), enlazados secuencialmente.

Implementación con cabeceras: Representa la matriz dispersa con una lista de cabeceras de filas y una de cabeceras de columnas; cada nodo no nulo tiene punteros al siguiente en su fila y al siguiente en su columna.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada la implementación de tres algoritmos de ordenamiento básicos sobre arreglos (Inserción, Selección y Burbuja), la aplicación de búsqueda lineal y binaria, y la representación eficiente de una Matriz Dispersa mediante dos enfoques: lista enlazada simple y lista con cabeceras.

La clase se orienta a que el estudiante comprenda la lógica de cada algoritmo trazándolo paso a paso, identifique su complejidad temporal y espacial, y reconozca en qué escenarios es preferible cada uno. Para la Matriz Dispersa, el estudiante debe comprender por qué almacenar únicamente los valores no nulos es más eficiente en memoria y cómo recorrerla.

4.Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

El ejercicio consiste en implementar manualmente en Java, sin usar métodos de ordenamiento de librerías (`Arrays.sort`, `Collections.sort`), los algoritmos Insertion Sort, Selection Sort y Bubble Sort sobre un arreglo de enteros. Se trazará paso a paso la ejecución de cada uno con el arreglo `[8, 4, 6, 3, 1, 9, 5]`. Adicionalmente, se implementará búsqueda lineal y binaria, y se construirá una Matriz Dispersa 5×5 con cabeceras que represente la siguiente matriz:

```
None
9  0  0  0  0
0  0  8  0  0
0  0  0  0  0
0  0  0  0  0
7  0  0  0  3
```

4.2 OBJETIVO DE LA DEMOSTRACIÓN

Demostrar el procedimiento para implementar manualmente los algoritmos de ordenamiento Inserción, Selección y Burbuja en Java, aplicar búsqueda lineal y binaria sobre el arreglo resultante, y construir una Matriz Dispersa con cabeceras, con el fin de que el estudiante comprenda la diferencia de eficiencia entre los algoritmos, sepa elegir el apropiado según el contexto, y comprenda cómo reducir el uso de memoria al representar matrices con pocos valores significativos.

4.3 RECURSOS NECESARIOS

- **Software:** Java JDK 11+, IDE (IntelliJ IDEA / Eclipse / VS Code)
- **Herramientas técnicas:** Compilador javac, terminal/consola
- **Materiales de apoyo:** Diapositivas Semana 6, pizarrón para trazar los pasos manualmente, tabla de complejidades proyectada
- **Archivos:** Archivo base `Main.java` con el arreglo de prueba ya declarado
- **Configuración mínima:** Java 11+, 4 GB RAM, proyector

4.4 DESARROLLO PASO A PASO

Paso 1 — Introducción y elección de algoritmo (10 min)

Mostrar la tabla de complejidades de la diapositiva 6. Plantear las tres preguntas clave: ¿Qué tan grande es la colección? ¿Cuánta memoria hay disponible? ¿La colección necesita crecer? Explicar que para colecciones pequeñas los algoritmos $O(n^2)$ son aceptables; para grandes volúmenes se necesitan $O(n \log n)$. El estudiante debe observar que Bubble Sort y Insertion Sort tienen $O(n)$ en el mejor caso (arreglo ya ordenado).

Paso 2 — Insertion Sort (20 min)

Trazar en pizarrón con [8, 4, 6, 3, 1, 9, 5]. Luego codificar:

java

```
Java
void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Qué observar: `key` guarda el elemento a insertar; el `while` desplaza elementos mayores una posición a la derecha. Al salir, `j+1` es la posición correcta.

Paso 3 — Selection Sort (15 min)

Trazar con el mismo arreglo. Luego codificar:

java

Java

```
void selectionSort(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        int temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}
```

Qué observar: Siempre hace $n-1$ comparaciones por pasada sin importar si el arreglo está ordenado $\rightarrow O(n^2)$ en todos los casos.

Paso 4 — Bubble Sort (15 min)

Trazar con el mismo arreglo. Luego codificar con optimización de flag:

java

Java

```
void bubbleSort(int[] arr) {
    boolean swapped;
    for (int i = 0; i < arr.length - 1; i++) {
        swapped = false;
        for (int j = 0; j < arr.length - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
    }
}
```

```
    }  
    if (!swapped) break; // arreglo ya ordenado  
  }  
}
```

Qué observar: El flag `swapped` permite salir temprano si el arreglo queda ordenado antes de terminar todas las pasadas → $O(n)$ en el mejor caso.

Paso 5 — Búsqueda Lineal y Binaria (10 min)

Sobre el arreglo ya ordenado [1, 3, 4, 5, 6, 8, 9]:

java

```
Java  
// Búsqueda Lineal -  $O(n)$   
int busquedaLineal(int[] arr, int objetivo) {  
    for (int i = 0; i < arr.length; i++)  
        if (arr[i] == objetivo) return i;  
    return -1;  
}  
  
// Búsqueda Binaria -  $O(\log n)$  - requiere arreglo ordenado  
int busquedaBinaria(int[] arr, int objetivo) {  
    int izq = 0, der = arr.length - 1;  
    while (izq <= der) {  
        int mid = (izq + der) / 2;  
        if (arr[mid] == objetivo) return mid;  
        else if (arr[mid] < objetivo) izq = mid + 1;  
        else der = mid - 1;  
    }  
    return -1;  
}
```

Paso 6 — Matriz Dispersa con Cabeceras (20 min)

Construir la estructura para la matriz del enunciado. Mostrar que de 25 celdas solo 4 tienen valor distinto de cero → 84% de desperdicio con matriz densa.

java

```
Java
class NodoMatriz {
    int valor, fila, columna;
    NodoMatriz siguienteFila, siguienteColumna;
}
class CabeceraFila { int numFila;  NodoMatriz primero;
CabeceraFila  sig; }
class CabeceraCol { int numCol;   NodoMatriz primero; CabeceraCol
sig; }
class MatrizDispersa {
    CabeceraFila  filas;
    CabeceraCol   columnas;
    void insertar(int val, int f, int c) { /* enlazar en fila y
columna */ }
}
```

Qué observar: Cada nodo se enlaza dos veces — una en su lista de fila y otra en su lista de columna. Insertar en orden mantiene las listas ordenadas para recorrido eficiente.

Observaciones Importantes

- **Insertion Sort** es el más eficiente de los tres para arreglos pequeños o casi ordenados. Es el algoritmo que usan internamente Timsort y algunos sistemas embebidos para subproblemas pequeños.
- **Selection Sort** siempre hace el mismo número de comparaciones independientemente del estado inicial del arreglo — no tiene optimización posible.
- **Bubble Sort sin flag** es el peor de los tres en práctica. Con el flag de **swapped** se vuelve $O(n)$ para arreglos ya ordenados.
- **Búsqueda binaria** solo funciona si el arreglo está ordenado. Aplicarla sobre un arreglo desordenado produce resultados incorrectos.

- En la Matriz Dispersa, olvidar enlazar el nodo en la lista de columna (solo enlazarlo en fila) es el error más común.
-

4.5 Tabla Comparativa — Pila vs Cola

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio	Estable
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Búsqueda Lineal	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—
Búsqueda Binaria	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—

4.6 Resultado Esperado

Operación	Resultado
Insertion Sort [8, 4, 6, 3, 1, 9, 5]	[1, 3, 4, 5, 6, 8, 9]
Selection Sort [8, 4, 6, 3, 1, 9, 5]	[1, 3, 4, 5, 6, 8, 9]
Bubble Sort [8, 4, 6, 3, 1, 9, 5]	[1, 3, 4, 5, 6, 8, 9]
Búsqueda binaria del 6 en arreglo ordenado	Retorna índice 4

Búsqueda lineal del 9

Retorna índice 5

Matriz Dispersa — insertar
(9,1,1),(8,2,3),(7,5,1),(3,5,5)

4 nodos enlazados, 21 celdas cero no
almacenadas

4.7 Aplicación Práctica

Los algoritmos de ordenamiento son la base de casi cualquier sistema de información: ordenar registros de pacientes, productos en un catálogo, resultados de búsqueda. La elección correcta impacta directamente el rendimiento; por eso el curso de cátedra mostró la tabla de complejidades comparativas.

La Matriz Dispersa es fundamental en el proyecto del curso (Cinema/MedTrack): el mapa de asientos por función es una matriz donde la mayoría de celdas están vacías. Implementarla como matriz densa (arreglo 2D completo) desperdicia memoria; implementarla como Matriz Dispersa con cabeceras solo almacena los asientos ocupados o reservados.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- La diferencia entre Insertion Sort y Selection Sort confunde a los estudiantes: uno inserta en la posición correcta; el otro selecciona el mínimo. Usar dos colores en el pizarrón (parte ordenada / desordenada) ayuda mucho.
- En Bubble Sort, los estudiantes olvidan que el índice `j` debe llegar hasta `arr.length - 1 - i` (no hasta `arr.length - 1`) porque los últimos `i` elementos ya están en su posición final.
- La Matriz Dispersa con cabeceras es el punto de mayor dificultad: entender que cada nodo se enlaza en DOS listas simultáneamente.

Recomendaciones:

- Trazar los tres algoritmos en pizarrón ANTES de abrir el IDE. 10 minutos de trazado manual ahorran 20 minutos de debugging.
- Preparar el arreglo de prueba `[8, 4, 6, 3, 1, 9, 5]` en un archivo `Main.java` base para no perder tiempo escribiendo setup.

- Para la Matriz Dispersa, dibujar el diagrama de la diapositiva 12 (implementación con cabeceras) en el pizarrón antes de codificar.
- Dejar la Búsqueda Binaria para después de ordenar el arreglo con alguno de los tres algoritmos — así el estudiante ve la conexión natural.

Puntos de mayor dificultad:

- El índice `j+1` en Insertion Sort al salir del while.
- El flag `swapped` en Bubble Sort optimizado.
- Doble enlace de nodos en la Matriz Dispersa (fila Y columna).
-

6. Referencias

1. GeeksforGeeks. (2023). Insertion Sort. <https://www.geeksforgeeks.org/insertion-sort/>
2. GeeksforGeeks. (2023). Selection Sort. <https://www.geeksforgeeks.org/selection-sort/>
3. GeeksforGeeks. (2023). Bubble Sort. <https://www.geeksforgeeks.org/bubble-sort/>
4. GeeksforGeeks. (2023). Sparse Matrix. <https://www.geeksforgeeks.org/sparse-matrix-representation/>
5. Cormen, T. H. et al. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Cap. 2: Ordenamiento por inserción y análisis de algoritmos.
6. Material de cátedra: Diapositivas Semana 6, EDD [B] — Arreglos y Matrices. USAC, 2026.